Code Architecture and Execution Flow in VibeVolts

Simulation Reference Documentation

July 25, 2026

1 Introduction

This document describes the high-level architecture, module-to-module execution flow, and state mutations within the VibeVolts space environment simulation toolkit. The simulation is structured as a discrete event model centered around a unified state dictionary called `sim_data` (instantiated as a `SimulationState` object).

2 System Architecture & Module Roles

VibeVolts is designed around modular components that initialize, propagate, schedule, scan, and process data. The key modules and their roles are:

- `minimalsimulation.py`: Defines the core data container class (`SimulationState`) inheriting from a custom `DotDict` to support both attribute-style and dict subscript access.
- `propagation.py`: Reads orbital elements (TLEs) and propagates satellite coordinates.
- `celestialbodies.py`: Resolves positions of key planetary bodies (Sun, Moon) using ephemerides.
- `targets.py`: Generates target coordinates (fixed reference points) on a logarithmic-radial Fibonacci sphere.
- `cadenceController.py`: Acts as the master orchestrator, advancing the time step and scheduling scans according to detector integration groups.
- `pointing.py`: Manages pointing grids and updates sensor orientations, skipping angles in exclusion zones.
- `exclusion.py`: Determines line-of-sight obstruction from the Earth, Moon, and Sun.
- `scandetectors.py`: Performs geometrical line-of-sight matching and calls radiometric models to calculate detection metrics.
- `dataHandling.py`: Accumulates detection logs into structured DataFrames for export and analysis.

3 Execution Flow

The simulation execution is divided into three main phases: Initialization, Configuration, and the Update/Scan Loop.

3.1 Phase 1: Initialization

1. `create_empty_simulation(start_time, delta_time)` initializes the `SimulationState`.
2. Variables initialized:
   - `sim_data.start_time`, `sim_data.time` → `datetime` (UTC).
   - `sim_data.counts` → empty `CountsState` (all asset counts initialized to 0).
   - `sim_data.pointing_spheres` → empty `dict`.

3.2 Phase 2: Configuration

In this phase, assets, detectors, and targets are added to `sim_data`:

1. **Satellites:** `add_satellites_from_tle()` loads a TLE file. It increments `sim_data.counts.satellites` and creates a `SatellitesState` with fields like `position`, `velocity`, `orbital_elements`, and `epochs`.
2. **Detectors:** A blank `DetectorArray` is initialized and appended to `sim_data.detector`. Specific sensor parameters (FOV, aperture, zero points, integration times) are populated via `makeDetector()`.
3. **Targets:** `add_fixed_points()` calls `generate_log_spherical_points()` to populate `sim_data.fixedpo` with reference coordinates, albedos, and sizes.
4. **Celestial Bodies:** `add_celestial_bodies()` allocates the `CelestialState` container for the Sun and Moon coordinates.

3.3 Phase 3: Update and Scan Loop

Once configured, the simulation loop is driven by the cadence controller:

1. `initCadence()` groups detectors by unique integration times into a sorted list of `CadenceGroup` records, setting the initial next scan times (`scanNext`) to `sim_data.time`.
2. The simulation update loop calls `nextIntegration()` repeatedly:
   - **Time Update:** Time is advanced directly to the next scheduled event: `sim_data.time = cadence.nextTime`.
   - **Orbit Propagation:** `propagate_satellites()` computes the new coordinates of all satellites at the updated time, writing to `sim_data.satellites.position`.
   - **Celestial Ephemeris:** `celestial_update()` queries Astropy ephemerides and writes coordinates to `sim_data.celestial.position`.
   - **Sensor Pointing:** `update_detector_pointing()` steps active detectors through their pointing sequence spheres, querying `exclusion()` to skip obscured angles, writing to `sim_data.detector.pointing`.
   - **Observation Scan:** `scandetectors()` is executed with the active group's mask. Visible targets are filtered via FOV geometry, and detection metrics (`signal`, `noise`, `snr`) are calculated.
   - **Rescheduling:** The active group's next scan is pushed forward by its interval: `group.scanNext += group.scanInterval`. The next global minimum event is recalculated.
   - **Data Collection:** Scan outputs are collected in the `DataHandler` instance for final DataFrame assembly.

4 Variable Mutation Lifecycle

The table below traces where the main parameters within the global `sim_data` are defined, mutated, and read.

Variable Field	Initialized By	Updated By	Primary Reader(s)
`sim_data.time`	`create_empty_sim`	`nextIntegration()`	Propagation, Ephemeris
`satellites.position`	`add_satellites_from_tle`	`propagate_satellites()`	`scandetectors()`, `exclusion`
`celestial.position`	`add_celestial_bodies`	`celestial_update()`	`scandetectors()`, `exclusion`
`detector.pointing`	`makeDetector()`	`update_detector_pointing()`	`scandetectors()`, `exclusion`
`detector.pointing_state`	`detectorPointingInitialize`	`update_detector_pointing()`	Pointing scheduler
`fixedpoints.position`	`add_fixed_points()`	*Static during run*	`scandetectors()`
`cadenceStructure`	`initCadence()`	`nextIntegration()`	Time advancement scheduler

Table 1: Lifecycle of primary state fields inside `sim_data`.